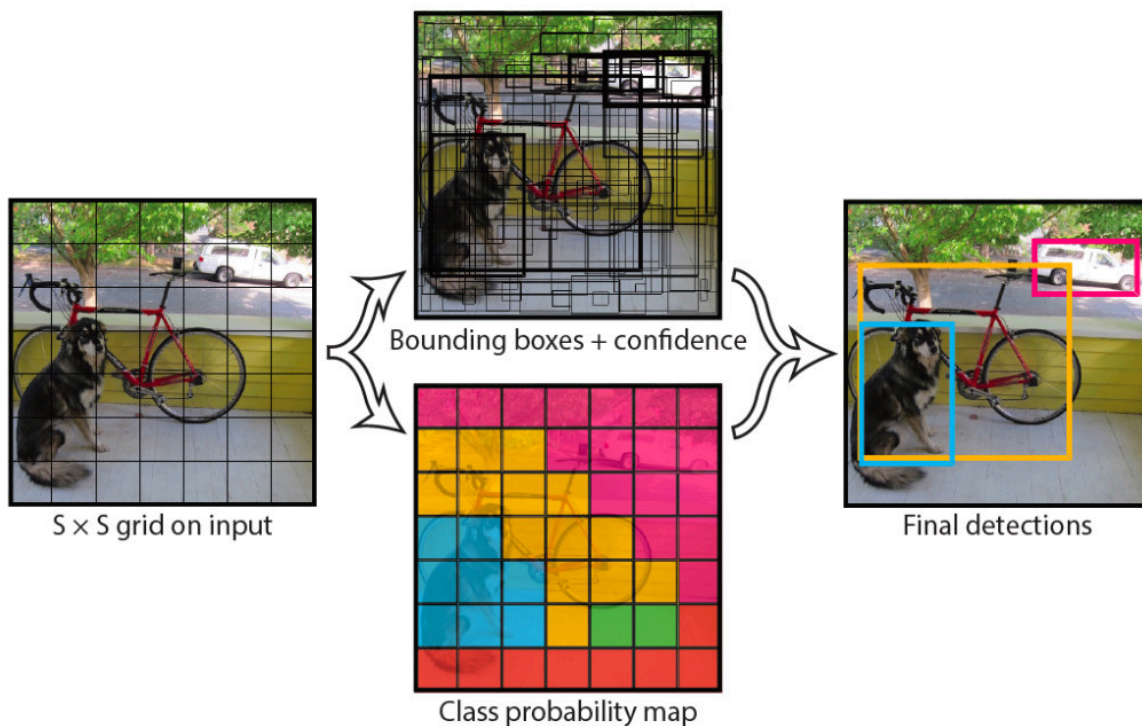


YOLO

2-stage detector는 localization과 classification을 수행하는 network 혹은 컴포넌트가 분리되어 있습니다. 이는 각 task가 순차적으로 진행되는 것을 의미하며, 이러한 과정에서 병목현상이 발생하여 detection 속도가 느려지게 됩니다. 반면 1-stage detector는 하나의 통합된 네트워크가 두 task를 동시에 진행합니다. YOLO v1은 대표적인 1-stage detector로, FPS를 개선하여 real-time에 가까운 detection 속도를 보였습니다.

Preview

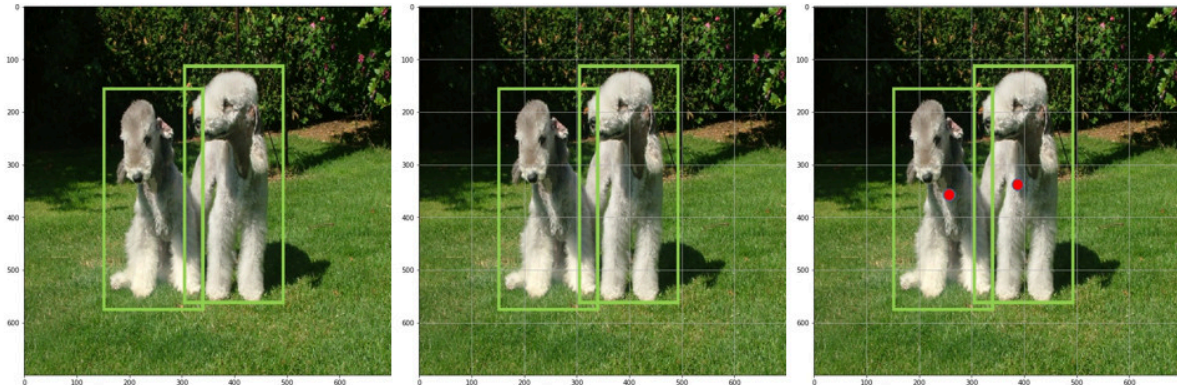


YOLO v1

YOLO v1은 localization과 classification을 하나의 문제로 정의하여 network가 동시에 두 task를 수행하도록 설계하였습니다. 이를 위해 이미지를 지정한 grid로 나누고, 각 grid cell이 한번에 bounding box와 class 정보라는 2가지 정답을 도출하도록 만들었습니다. 또한 각 grid cell에서 얻은 정보를 잘 feature map이 잘 encode할 수 있도록 독자적인 Convolutional Network인 DarkNet을 도입합니다. 이를 통해 얻은 feature map을 활용하여 자체적으로 정의한 regression loss를 통해 전체 모델을 학습시킵니다.

Main Ideas

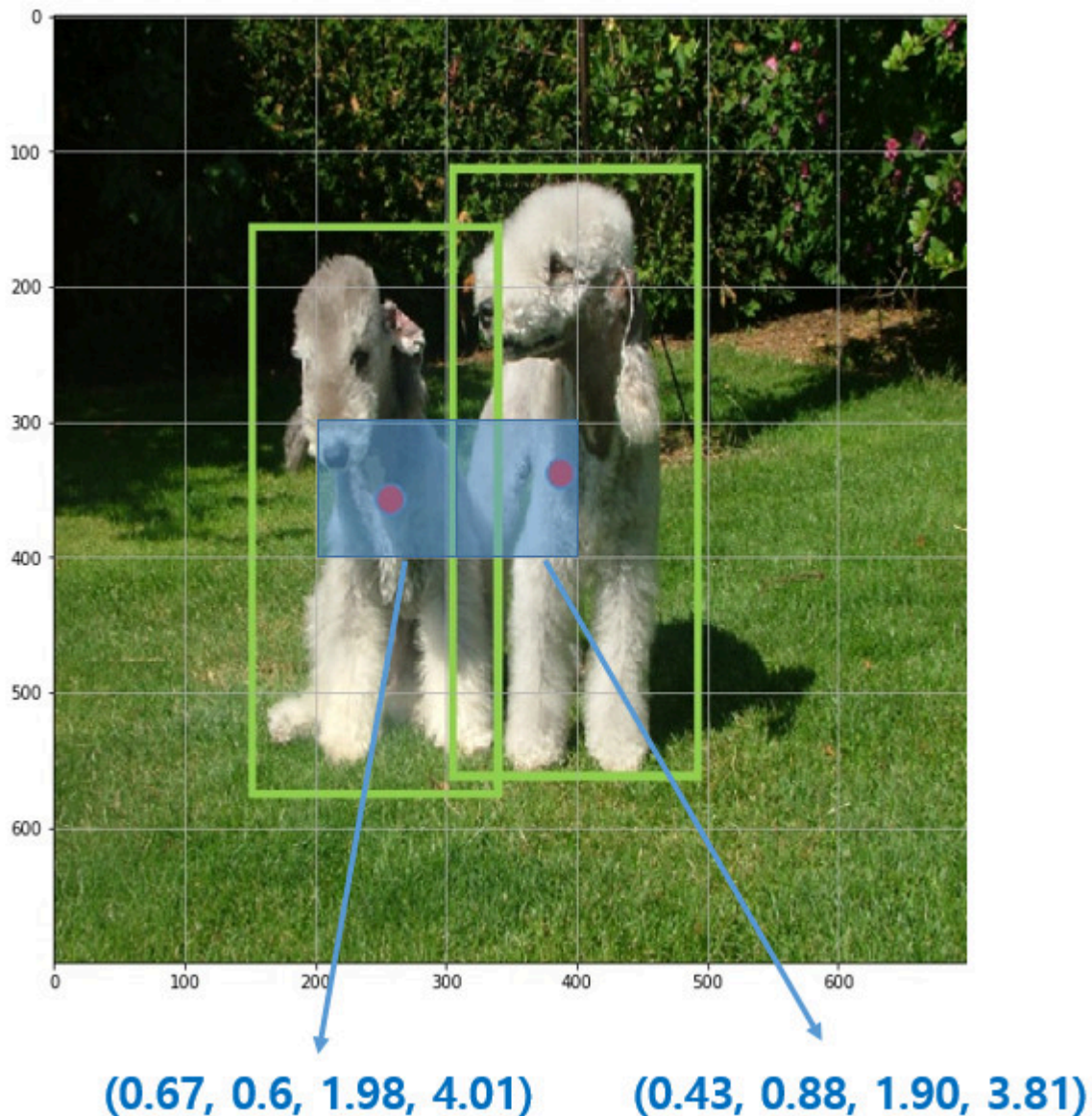
1) 1-stage detector



divide images by grid $S \times S$

YOLO v1은 별도의 region proposals를 사용하지 않고 전체 이미지를 입력하여 사용합니다. 먼저 **전체 이미지를 $S \times S$ 크기의 grid로 나눕니다**. 여기서 객체의 중심이 특정 grid cell에 위치한다면, 해당 grid cell은 그 객체를 detect하도록 **할당(responsible for)**됩니다. 위의 그림을 보면 4행 3열의 grid cell이 왼쪽의 개를 예측하도록 할당되었고, 4행 4열의 grid cell이 오른쪽의 개를 예측하도록 할당되었습니다. 이는 곧 나머지 grid cell은 객체를 예측하는데 참여할 수 없음을 의미합니다.

각각의 grid cell은 B개의 bounding box와 해당 bounding box에 대한 confidence score를 예측합니다. confidence score는 해당 bounding box에 객체가 포함되어 있는지 여부와, box가 얼마나 정확하게 ground truth box를 예측했는지를 반영하는 수치입니다. confidence score는 $\text{Pr}(\text{Object}) * \text{IoU}(\text{truthpred})$ 로 정의합니다. 만약 grid cell 내에 객체가 존재하지 않는다면 confidence score는 0이 될 것입니다. 반대로 grid cell 내에 객체가 존재한다면 confidence score는 IoU 값과 같아집니다.



YOLO format bounding box coordinates

각각의 **bounding box**는 **box의 좌표 정보(x, y, w, h)**와 **confidence score**라는 5개의 예측값을 가집니다. 여기서 (x, y)는 grid cell의 경계에 비례한 box의 중심 좌표를 의미합니다. 높이와 너비는 역시 마찬가지로 grid cell에 비례한 값입니다. 여기서 주의할 점은 x, y는 grid cell 내에 위치하기에 0~1 사이의 값을 가지지만 객체의 크기가 grid cell의 크기보다 더 클 수 있기 때문에 width, height 값은 1 이상의 값을 가질 수 있습니다.

하나의 **bounding box**는 하나의 객체만을 예측하며, 하나의 **grid cell**은 하나의 **bounding box**를 학습에 사용합니다. 예를 들어 grid cell별로 B개의 bounding box를 예측한다고 할 때, confidence score가 가장 높은 1개의 bounding box만 학습에 사용하는 것입니다.

$$pred_{cell} = [\underbrace{c_1, c_2, \dots, c_{20}}_{\text{Class probabilities}}, \underbrace{p_{c_1}}_{\text{Box1 confidence score}}, \underbrace{x, y, w, h}_{\text{Box 1}}, \underbrace{p_{c_2}}_{\text{Box2 confidence score}}, \underbrace{x, y, w, h}_{\text{Box 2}}]$$

Prediction per grid cell

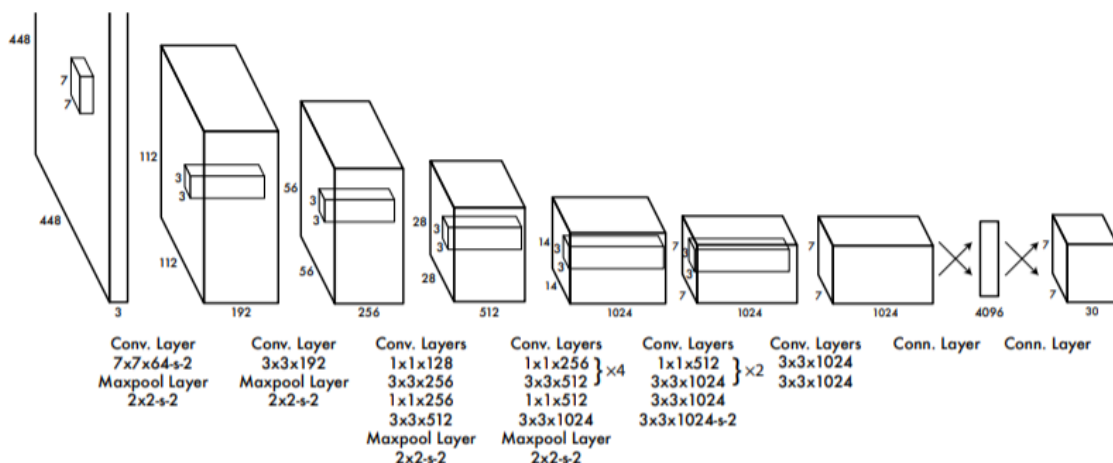
각 grid cell은 C개의 **conditional class probabilities**

인 $\Pr(\text{Class}|\text{Object})$ 를 예측합니다. 이는 특정 grid cell에 객체가 존재한다고 가정했을 때, 특정 class i일 확률인 조건부 확률값입니다. bounding box 수와 상관없이 하나의 grid cell마다 하나의 조건부 확률을 예측합니다. 여기서 짚고 넘어갈 점은 bounding box별로 class probabilities를 예측하는 것이 아니라 grid cell별로 예측하는 것입니다.

논문에서는 $S=7, B=2, C=20$ 설정하였습니다(PASCAL VOC 데이터셋을 사용하여 학습하였기에 class의 수가 20개). 즉, 이미지를 7x7 grid로 나누고 각 grid cell은 2개의 bounding box와 해당 box의 confidence score, 그리고 C개의 class probabilities를 예측합니다. 즉 **이미지별 예측값의 크기는 $7 \times 7 \times (2 \times 5 + 20)$ 입니다.** 이와 같은 과정을 통해 bounding box의 위치와 크기, 그리고 class에 대한 정보를 동시에 예측하는 것이 가능해 집니다.

2) DarkNet

YOLO v1 모델은 앞서 살펴본 최종 예측값의 크기인 $7 \times 7 \times 30$ 에 맞는 feature map을 생성하기 위해 **DarkNet**이라는 독자적인 Convolutional Network을 설계합니다. 전체 network의 구조는 아래와 같습니다.



DarkNet

DarkNet은 ImageNet 데이터셋을 통해 학습시켰습니다. 학습 결과 GoogLeNet 모델과 비슷한 top-5 88% 정도의 정확도를 보입니다. 이후 모델이 detection task를 수행할 수 있도록 4개의 conv layer와 2개의 fc layer를 추가합니다. 또한 classification task를 위해 학습시켰을 때는 224x224 크기의 이미지를 사용한 반면, detection task를 위한 학습 시에는 이미지의 크기를 키워 448x448 크기의 이미지를 사용한다고 합니다. 이는 detection task는 결이 고운(fine grained) 시각 정보를 필요로 하기 때문이라고 설명합니다.

3) Loss function

Localization loss

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

Confidence loss

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2$$

Classification loss

$$+ \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

YOLO v1 loss function

기존 R-CNN 계열의 모델이 classification, localization task에 맞게 서로 다른 loss function을 사용했던 것과 달리 YOLO v1 모델은 regression 시 주로 사용되는 **SSE(Sum of Squared Error)**를 사용합니다. 위의 그림에서 볼 수 있듯이 Localization loss, Confidence loss, Classification loss의 합으로 구성되어 있습니다.

Localization loss

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

Localization loss

- **λ_{coord}** : 많은 grid cell은 객체를 포함하지 않아 confidence score가 0이 되어 객체를 포함하는 grid cell의 gradient를 압도하여, 모델이 불안정해질 수 있습니다. λ_{coord} 는 이러한 문제를 해결하기 위해 객체를 포함하는 cell에 가중치를 두는 파라미터입니다. 논문에서는 $\lambda_{\text{coord}}=5$ 로 설정합니다.

λ_{coord}

$\lambda_{\text{coord}}=5$

- S^2 : grid cell의 수(=7x7=49)

S^2

- B : grid cell별 bounding box의 수(=2)

B

- $\mathbb{1}_{ij}$: i번째 grid cell의 j번째 bounding box가 객체를 예측하도록 할당 (responsible for)받았을 때 1, 그렇지 않을 경우 0인 index parameter입니다. 앞서 설명했듯이 grid cell에서는 B개의 bounding box를 예측하지만 그 중 confidence score가 높은 오직 1개의 bounding box만을 학습에 사용합니다.

$\mathbb{1}_{ij}$

i

j

- x_i, y_i, w_i, h_i : ground truth box의 x, y 좌표와 width, height. 여기서 크기가 큰 bounding box의 작은 오류가 크기가 작은 bounding box의 오류보다 덜 중요하다는 것을 반영하기 위해 w_i, h_i 값에 루트를 씌어주게 됩니다.

x_i, y_i, w_i, h_i

w_i, h_i

- $\hat{x}_i, \hat{y}_i, \hat{w}_i, \hat{h}_i$: 예측 bounding box의 x, y 좌표, width, height

$\hat{x}_i, \hat{y}_i, \hat{w}_i, \hat{h}_i$

Confidence loss

$$\begin{aligned}
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2 \\
 & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2
 \end{aligned}$$

Confidence loss

- λ_{noobj} : 앞서 언급한 객체를 포함하지 않는 grid cell에 곱해주는 가중치 파라미터입니다. 논문에서는 $\lambda_{\text{noobj}}=0.5$ 로 설정했습니다. $\lambda_{\text{obj}}=5$ 로 설정한 것에 비해 상당히 작게 설정하여 객체를 포함하지 않은 grid cell의 영향력을 줄였습니다.

λ_{noobj}

$\lambda_{\text{noobj}}=0.5$

$\lambda_{\text{obj}}=5$

- $\mathbb{1}_{i,j}^{\text{noobj}}$: i 번째 grid cell의 j 번째 bounding box가 객체를 예측하도록 할당 (responsible)받지 않았을 때 1, 그렇지 않을 경우 0인 index parameter입니다.

$\mathbb{1}_{i,j}^{\text{noobj}}$

i

j

- C_i : 객체가 포함되어 있을 경우 1, 그렇지 않을 경우 0

C_i

- $\hat{C}_i$: 예측한 bounding box의 confidence score

$\hat{C}_i$

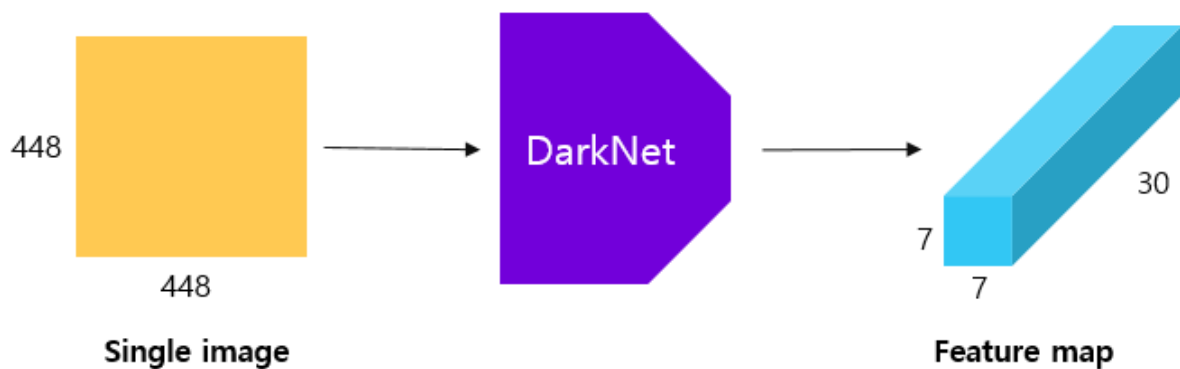
Classification loss

$$+ \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

Classification loss

- $p_i(c)$: 실제 class probabilities
 $p_i(c)$
- $\hat{p}_i(c)$: 예측한 class probabilities
 $\hat{p}_i(c)$

Training YOLO v1



Training YOLO v1

YOLO v1 모델의 학습 과정은 굉장히 단순합니다. 앞서 정의한 DarkNet에 이미지를 입력하여 7x7x30 크기의 feature map을 loss function을 통해 학습시킵니다.

Detection

detection 시에는 최종 예측 결과에 **Non Maximum Suppression** 알고리즘을 적용합니다. 이를 통해 mAP 값이 2~3% 정도의 향상된다고 합니다.

- YOLO v1 모델은 base network의 경우 45fps, 경량화한 fast version의 network는 150fps의 결과를 보여 매우 빠른 detection 속도를 보입니다. 실시간으로 0.0025 이하의 지연시간(latency)를 가지며 객체를 detect하는 것이 가능하다고 합니다.
- 또한 sliding window 방식이나 region proposal 기반의 모델과는 달리 YOLO v1 모델은 전체 이미지를 인지하여 맥락 정보(contextual information)을 학습한다고 합니다.

다. 이를 통해 배경 영역을 객체로 인식하는 False Positive 오류를 Fast R-CNN 모델보다 상대적으로 덜 범한다고 합니다.

- YOLO v1 모델은 일반화 가능한 표현(representations)을 학습하여 새로운 도메인이거나 예상치 못한 입력 이미지에 대해 상대적으로 강건한 모습을 보입니다.

하지만 당시 state-of-the-art 모델보다 정확도 측면에서 약간 부족한 성능을 보입니다. 특히 작은 객체를 제대로 탐지하지 못하는 단점이 있습니다. 또한 각 grid cell의 bounding box는 하나의 객체만을 예측하기 때문에 같은 grid cell 내에 있는 여러 객체를 탐지하지 못합니다.